TensorFlow & Keras 3 Cheat Sheet

Python 3.13, TensorFlow 2.21 with Keras 3.15 (2.22 upcoming, not covered), checked October 2026

Install matrix

CPU / default	`pip install tensorflow`
NVIDIA, Linux / WSL2	`pip install 'tensorflow[and-cuda]'`
CPU slim	`tensorflow-cpu`
macOS Apple GPU	`tensorflow-metal` plug-in
Native Windows	CPU only after 2.10; use WSL2
Keras 2	`tf_keras` (2.15)
Docker	`tensorflow/tensorflow:latest-gpu`

Python 3.10-3.13; TensorFlow 2.21 + Keras 3.15. No conda.

Tensors and variables

```
a = tf.constant([[1., 2.], [3., 4.]])
v = tf.Variable(tf.zeros([2, 2]))
v.assign_add(a)
tf.reshape(a, [-1]); a.numpy()
tf.config.list_physical_devices("GPU")
```

- Tensors are immutable; `Variable` holds state
- Default dtype float32; ints are int32
- Ragged and sparse: `tf.ragged`, `tf.sparse`

GradientTape

```
with tf.GradientTape() as tape:
    pred = model(x, training=True)
    loss = loss_fn(y, pred)
grads = tape.gradient(loss,
model.trainable_variables)
opt.apply_gradients(zip(grads,
model.trainable_variables))
```

- Tape is freed after one `gradient` call; `persistent=True` to reuse
- `tape.watch` for plain tensors
- `None` gradient = disconnected graph

tf.function rules

- `@tf.function` traces Python once per input signature
- Python side effects (print, list append) run only at trace time
- Use `tf.print`, `tf.cond`, `tf.while_loop` for dynamic logic
- Create `tf.Variable` only on the first call
- Pass tensors, not Python numbers, to avoid retracing
- `input_signature=[tf.TensorSpec(...)]` fixes the trace
- `jit_compile=True` enables XLA

Keras 3 model styles

```
import keras
L = keras.layers
m = keras.Sequential([L.Dense(64, "relu"),
                      L.Dense(3)])
i = keras.Input((10,)); o = L.Dense(3)(i)
m = keras.Model(i, o)       # functional
class Net(keras.Model): ... # subclass
```

Backends: TensorFlow, JAX, PyTorch (`KERAS_BACKEND`).

compile and fit

```
model.compile(
  optimizer=keras.optimizers.AdamW(1e-3),
  loss=keras.losses.
   SparseCategoricalCrossentropy(
    from_logits=True),
  metrics=["accuracy"])
h = model.fit(train_ds,
validation_data=val_ds,
              epochs=20, callbacks=cbs)
model.evaluate(test_ds); model.predict(x)
```

Callbacks

`EarlyStopping`	`patience`, restore best weights
`ModelCheckpoint`	`.keras`, best only
`ReduceLROnPlateau`	factor 0.1-0.5
`LearningRateScheduler`	per-epoch lr
`TensorBoard`	`log_dir`, histograms
`CSVLogger`, `BackupAndRestore`	logs, resume

tf.data recipe

```
AT = tf.data.AUTOTUNE
ds =
(tf.data.Dataset.from_tensor_slices((x, y))
      .shuffle(10_000)
      .map(prep, num_parallel_calls=AT)
      .batch(64)
      .prefetch(AT))
```

- Shuffle before batch; `cache()` after cheap maps
- Fixed batch shapes for TPUs: `drop_remainder=True`
- Keras preprocessing layers can live inside the model

Saving formats

`model.save("m.keras")`	full model; Keras 3 default
`save_weights("m.weights.h5")`	weights only
`model.export("dir")`	SavedModel for serving
`keras.saving.load_model`	loads `.keras`, safe mode
`tf.train.Checkpoint`	custom loops, object-based

Mixed precision and tf.distribute

```
keras.mixed_precision.set_global_policy(
    "mixed_bfloat16")   # or
"mixed_float16"
s = tf.distribute.MirroredStrategy()
with s.scope():
    model = build(); model.compile(...)
```

- float16 uses loss scaling; keep the output layer float32
- Scale global batch by replica count
- `MultiWorkerMirroredStrategy` + `TF_CONFIG`; `TPUStrategy`

LiteRT and Serving

LiteRT (ex-TF Lite)	`model.export("m.tflite", format="litert")` or `TFLiteConverter`; run with `ai-edge-litert`
Quantisation	post-training int8 / float16
TF Serving	`tensorflow/serving`, REST 8501, gRPC 8500, `/models/NAME/1`
TF.js	browser deployment

TF1 and Keras 2 to 3

`tf.Session`, `placeholder`	call functions; `@tf.function`
`get_variable`, scopes	`tf.Variable` in `tf.Module` / layers
`tf.keras.*`	`import keras`
`optimizers.legacy`	only via `tf_keras`; Keras 3 optimisers
`model.save("dir")` SavedModel	`.keras`; `TFSMLayer` to reuse SavedModel
`tf.contrib`, estimators	removed; use Keras

Generated with AI assistance; verify against official documentation. Full detail in the TensorFlow part of the docs site.